

Laboratorio 2026 de Programación Imperativa

Tarea 1

Información general

Se sugiere leer detenidamente todo el texto antes de comenzar a trabajar en esta tarea de laboratorio. Es muy importante que se respeten todos los requisitos solicitados en esta sección y en las siguientes. Si surgen dudas, pedimos que las formulen en el foro del laboratorio de la página EVA del curso.

Individualidad

Esta tarea deberá realizarse de forma **individual**. Para todas las tareas de laboratorio rige el [Reglamento del Instituto de Computación ante Instancias de No Individualidad en los Laboratorios](#) (lectura obligatoria).

Uso de IA

Con respecto al uso de IA, este curso adhiere a la categoría 1: Nivel de uso reducido. Está prohibido el uso de IA para generar parte o toda la solución de las tareas de evaluación asignadas por el equipo docente. Para más información leer [Sobre el uso de herramientas basadas en IA en Programación Imperativa](#).

Calendario

Entrega: La entrega de la tarea puede realizarse hasta **las 12:00 del mediodía del miércoles 16 de septiembre**. Los trabajos deberán entregarse dentro de los plazos establecidos. **No** se aceptarán trabajos fuera de plazo.

Reentrega: Todos los estudiantes que realizaron la entrega pueden hacer modificaciones y realizar una segunda entrega (*reentrega*). La reentrega puede realizarse **hasta las 20:00 del viernes 18 de septiembre**.

Forma de entrega

Se debe entregar un **único** archivo llamado `tarea1.pas` que debe contener **únicamente** el código de los subprogramas pedidos y, eventualmente, el de los subprogramas auxiliares que se necesiten para implementarlos. La entrega se realiza mediante una actividad en la plataforma EVA en la sección de laboratorio.

Archivos provistos y ejecución de pruebas

El archivo `definiciones.pas` contiene constantes (`SEPARADOR` y `FINALIZADOR`) definidas por los docentes. El programa principal para realizar pruebas es proporcionado por los docentes en el archivo `principal.pas`. **No se debe modificar ninguno de estos archivos, dado que no serán entregados**. Para usar este programa se debe leer y seguir las instrucciones provistas en la

sección “[cómo ejecutar los casos de prueba de la Primera Tarea](#)”. Los casos de prueba que se tomarán en cuenta para corregir la tarea son los que se especifican en esa sección.

Introducción

El Procesamiento de Lenguaje Natural (PLN) es un área de la informática que estudia técnicas para analizar, interpretar y procesar el lenguaje humano.

Una de las tareas de la disciplina es el análisis léxico de las oraciones, extrayendo información relevante a partir de su contenido. El análisis léxico es el proceso de examinar un texto para identificar y clasificar sus unidades básicas, como caracteres, palabras o símbolos, de acuerdo con determinadas reglas.

En este laboratorio se propone trabajar en la extracción y el reconocimiento de algunas características de una oración, en particular, la cantidad de ocurrencias de vocales y consonantes y el promedio de la longitud de las palabras.

Esta información podría utilizarse, por ejemplo, para intentar identificar el idioma en el que está escrito un texto, a partir de ciertos patrones estadísticos característicos de cada lengua.

Aclaraciones y simplificaciones

Se consideran las siguientes simplificaciones:

- Las oraciones solo contienen caracteres alfabéticos (pueden ser mayúsculas y minúsculas, pero sin tildes), separadores y finalizadores. Las oraciones **NO** contienen números, símbolos especiales ni la letra ‘ñ’.
- Todas las palabras de la oración estarán separadas por un único **SEPARADOR**. Las oraciones no comenzarán con **SEPARADOR**; solo podrán comenzar con una palabra o con **FINALIZADOR**.
- Una oración sin palabras también es válida.

Subprogramas

Se debe implementar el siguiente subprograma:

```
procedure conteoOracion(maxCantPalabras: integer;  
                        var cantA, cantE, cantI, cantO, cantU, cantConsonantes : integer;  
                        var oracionNula, superaMaximo : boolean;  
                        var largoPromedio : real);
```

conteoOracion lee de la entrada estándar una oración, es decir, una cadena de caracteres terminada en **FINALIZADOR**, donde cada palabra está separada por un **SEPARADOR**. Además, recibe en **maxCantPalabras** el largo máximo que puede tener la oración en palabras (**estrictamente mayor que cero**). Finalmente, retorna en los parámetros de salida:

- **cantX** ($X \in \{ 'A', 'E', 'I', 'O', 'U' \}$) la cantidad de ocurrencias de la vocal correspondiente.
- **cantConsonantes**: la cantidad de ocurrencias de consonantes.
- **oracionNula**: indica si la oración no tiene ninguna palabra.
- **superaMaximo**: indica si la oración tiene más palabras que **maxCantPalabras**.
- **largoPromedio**: contiene el promedio del largo de las palabras.

Si la cantidad de palabras de la oración supera **maxCantPalabras**, solo se procesan y se tienen en cuenta para el conteo las primeras **maxCantPalabras** palabras. En el caso de que la oración sea nula (es decir, que no tenga ninguna palabra), el valor en **largoPromedio** **no es relevante**.

Ejemplos

A continuación se muestran ejemplos de ejecución del programa principal, asumiendo que **SEPARADOR** es ' ' y **FINALIZADOR** es '.'

Ejemplo 1

```
-----
Ingrese la cantidad de palabras máxima de la oración: 40
Ingrese la oración: Este es un ejemplo.
-----
ANÁLISIS DE LA ORACIÓN
-----
¿Supera la cantidad máxima de palabras?
NO.
-----
¿La oración es nula?
NO (Largo promedio de palabra: 3.75)
-----
Ocurrencias de vocales:
A: 0
E: 5
I: 0
O: 1
U: 1
-----
Ocurrencias de consonantes:
Consonantes: 8
-----
```

Ejemplo 2

```
-----
Ingrese la cantidad de palabras máxima de la oración: 3
```

```

-----
Ingrese la oración: Este es Otro EJEMPLO A probaR.
-----
ANÁLISIS DE LA ORACIÓN
-----
¿Supera la cantidad máxima de palabras?
SI (Para el análisis se consideran las primeras 3 palabras)
-----
¿La oración es nula?
NO (Largo promedio de palabra: 3.33)
-----
Ocurrencias de vocales:
A: 0
E: 3
I: 0
O: 2
U: 0
-----
Ocurrencias de consonantes:
Consonantes: 5
-----

```

Ejemplo 3

```

-----
Ingrese la cantidad de palabras máxima de la oración: 1
Ingrese la oración: .
-----
ANÁLISIS DE LA ORACIÓN
-----
¿Supera la cantidad máxima de palabras?
NO.
-----
¿La oración es nula?
SI.
-----
Ocurrencias de vocales:
A: 0
E: 0
I: 0
O: 0
U: 0
-----
Ocurrencias de consonantes:
Consonantes: 0
-----

```

Aprobación

Para aprobar el curso, es necesario aprobar este laboratorio. Para aprobar este primer laboratorio, el archivo `tarea1.pas` debe compilar. En caso de entregar una tarea que compile, el equipo docente le brindará una retroalimentación con los casos de prueba que fueron exitosos y los que no y un análisis estático de las estructuras utilizadas. Esta retroalimentación es de vital importancia para entender los errores cometidos y poder mejorar como programadores. Además, será útil de cara al primer parcial, ya que habrá un ejercicio relacionado con la situación abordada en este laboratorio.

Se pide

Escribir un archivo con el subprograma solicitado. El encabezado del subprograma **debe coincidir exactamente** con el que figura en esta letra. Si el estudiante realiza algún cambio, se considerará que el subprograma no se implementó. Si el estudiante lo desea, puede implementar subprogramas auxiliares adicionales (además del subprograma solicitado).

Para la corrección, las tareas se compilarán con una versión igual o posterior a **3.0.4 para Linux**. La compilación y la ejecución se realizarán en la línea de comandos. El comando de compilación se invocará de la siguiente manera:

```
fpc -Co -Cr -Miso -gl principal.pas
```

`principal.pas` será el programa principal entregado por el equipo docente.

La entrega debe poder compilarse desde la línea de comandos **SIN** utilizar un IDE.

No está permitido utilizar facilidades de Free Pascal que no forman parte del estándar y no se dan en el curso. Así, por ejemplo, no se pueden utilizar las siguientes extensiones, unidades, tipos o instrucciones: `uses`, `clrscr`, `gotoxy`, `crt`, `readkey`, `longint`, `string`, `break`, `exit`, etcétera.

En esta tarea, como en todos los problemas de este curso, se valorará, además de la lógica correcta, el uso de un buen estilo de programación de acuerdo con los criterios impartidos en el curso. De esta manera, se hará énfasis en buenas prácticas de programación que lleven a un código legible, bien documentado y mantenible, tales como:

- indentación adecuada,
- utilización correcta y apropiada de las estructuras de control,
- código claro y legible,
- algoritmos razonablemente eficientes,
- utilización de comentarios que documenten y complementen el código,
- utilización de constantes simbólicas,
- nombres mnemotécnicos para variables, constantes, etcétera.

Para resolver la tarea se pueden utilizar todos los conocimientos vistos en el curso.